Name: _____
Surname: _____
Date: _____

Debugging (15 points each)

Instructions

The functions below contain one or more errors. You have to find these errors, explain why they happen, and suggest a fix to them.

Questions

1. The function below is designed to remove all vowels from a given string. However, it doesn't remove all the vowels as expected. Identify the problem and fix the code.

```
1  def remove_vowels(input_str: str) -> str:
2      vowels = ("a", "e", "i", "o", "u")
3      result = ""
4      for char in input_str:
5          if char.lower() in vowels:
6              input_str.replace(char, "")
7      return input_str
8
9  text = "Programming"
10 print(remove_vowels(text))  # Output: "Programming"
```

Answer:

The function never changes the string. The method `replace()` does not modify a string in place (strings are immutable): it **returns** a new string. In line 6 that new string is created and immediately thrown away, because we do not store it anywhere. *(+6 points)*

Because of that, the function ends by returning `input_str`, which is still the original text. The variable `result`, created in line 3, is never used. This is why the output is "Programming". *(+4 points)*

We should build the answer in `result`: for every character that is **not** a vowel (comparing its lowercase version, so that uppercase vowels are also removed), we add it to `result` with `result += char`. Finally, we return `result`. *(+5 points)*

(Optional, fixed code)

```
1  def remove_vowels(input_str: str) -> str:
2      vowels = ("a", "e", "i", "o", "u")
3      result = ""
4      for char in input_str:
5          if char.lower() not in vowels:
6              result += char
7      return result
```

2. The following function is supposed to merge two dictionaries, without modifying the originals. In case of key conflicts, the value from the second dictionary should overwrite the one from the first. The code raises an error on line 5: "`SyntaxError: cannot assign to function call here`". Identify and fix the issue(s).

```
1  def merge_dictionaries(dict1: dict, dict2: dict) -> dict:
2      merged_dict = dict1
3      for key, value in dict2.items():
4          merged_dict.keys() = value
5      return merged_dict
6
7  dict_a = {"a": 1, "b": 2}
8  dict_b = {"b": 3, "c": 4}
9  print(merge_dictionaries(dict_a, dict_b))
10 # Expected: {"a": 1, "b": 3, "c": 4}
```

Answer:

There are two errors. The first one is in line 5, the one that raises the `SyntaxError`. The method `merged_dict.keys()` returns the keys of the dictionary, and a function call cannot be on the left side of an assignment. To add a pair or replace the value of an existing key, we use the key as an index: `merged_dict[key] = value`. *(+7 points)*

The second error is in line 2. The command `merged_dict = dict1` does not create a new dictionary: both names refer to the **same** dictionary. Once the first error is fixed, every update would also modify `dict1` (which is `dict_a` outside the function), and the exercise asks us not to modify the originals. We need to create a copy with `dict1.copy()` (or `dict(dict1)`). *(+8 points)*

(Optional, fixed code)

```
1  def merge_dictionaries(dict1: dict, dict2: dict) -> dict:
2      merged_dict = dict1.copy()
3      for key, value in dict2.items():
4          merged_dict[key] = value
5      return merged_dict
```

3. This code is meant to generate a list of squares of numbers from 1 to 5 using a while loop. However, it results in an infinite loop. Find the mistake and correct it.

```python
def generate_squares(limit: int=5) -> list[int]:
    squares = []
    i = 1
    while i <= limit:
        squares.append(i ** 2)
    return squares

limit = -1
print(generate_squares())  # Expected: [1, 4, 9, 16, 25]
```

Answer:

The loop condition is `i <= limit`, but the variable `i` never changes inside the loop. It starts at 1 and stays at 1, so the condition is always true and the loop never ends. *(+7 points)*

We need to increase `i` at the end of every iteration, adding `i += 1` **inside** the loop (indented, after the `append`). If it is written outside the loop, it is never reached. *(+8 points)*

Comment: The line `limit = -1` in the global scope is irrelevant: the function uses its own parameter `limit`, whose default value is 5.

(Optional, fixed code)

```python
def generate_squares(limit: int=5) -> list[int]:
    squares = []
    i = 1
    while i <= limit:
        squares.append(i ** 2)
        i += 1
    return squares
```

4. You have created a function that stacks two 1D-arrays, one of top of another. If the arrays have a different length, the function fills that difference with zeros. Your function raises an error on line 6: "`ValueError: could not broadcast input array from shape (3,) into shape (0,)`". Find the mistake(s) and correct it.

```python
import numpy as np

def stack_uneven(x: np.array, y: np.array) -> np.array:
    if len(x) > len(y):
        y = np.zeros(len(x))
        y[len(y):] = y
    else:
        x = np.zeros(len(y))
        x[len(x):] = x
    return np.stack((x, y))

x = np.array([1, 2, 3])
y = np.array([4, 5])
print(stack_uneven(x, y))
```

```
# Expected:
# [[1, 2, 3],
#  [4, 5, 0]]
# Output:
# [[1., 2., 3.],
#  [0., 0., 0.]]
```

Answer:

The first problem is in lines 5 and 6. When `x` is longer, the code replaces `y` with a new array of zeros, so the original values of `y` are lost. Even if line 6 did not fail, the second row of the result would be only zeros, which is the (wrong) output shown. The same happens in the `else` branch with `x`. *(+5 points)*

The second problem is the slice in line 6. At that point `len(y)` is already 3, so `y[len(y):]` is `y[3:]`, an empty slice of shape `(0,)`. Assigning an array of shape `(3,)` to it is what raises the `ValueError`. We want to fill the **first** positions, that is, `[:len(original)]`. *(+5 points)*

To fix it, we create the array of zeros in a new variable, copy the original values into its first positions, and only then use it in place of the short array. The same is done for `x` in the `else` branch. *(+5 points)*

Comment: `np.zeros()` creates floats by default, so the result would show `4., 5., 0.`. Using `dtype=y.dtype` keeps the integers of the expected output.

(Optional, fixed code)

```
import numpy as np

def stack_uneven(x: np.array, y: np.array) -> np.array:
    if len(x) > len(y):
        padded = np.zeros(len(x), dtype=y.dtype)
        padded[:len(y)] = y
        y = padded
    else:
        padded = np.zeros(len(y), dtype=x.dtype)
        padded[:len(x)] = x
        x = padded
    return np.stack((x, y))
```

Pseudo-Code

Instructions

In the following exercises, you will be asked to write Python functions by hand. Please follow these guidelines:

- Any build-in function you need is in the cheat-sheet provided.
- Clearly indicate each level of indentation using a right arrow (⇒).
- Ensure your code works correctly for any possible example.
- If your code has errors, it will be evaluated based on the provided docstring, comments, and explanations.

Full points will be awarded for code that performs as expected under all conditions.

5. Greatest Common Divisor (20 points)

The Greatest Common Divisor (GCD) of two integers is the largest positive integer that divides each of the integers without leaving a remainder. For instance, the GCD of 6 and 15 is 3. Write a Python function named `find_gcd()` that takes two integers as input and returns the GCD of these numbers. *(12 points)*

- Your function should handle cases where either or both of the input numbers could be negative, returning the GCD as a positive integer. *(3 points)*
- If one of the inputs is zero, the GCD should be the absolute value of the non-zero number. For example, the GCD of 0 and 5 is 5. *(3 points)*
- If both inputs are zero, the function should return 0, as GCD is not defined in this case. *(2 points)*

Answer:

A good approach is the Euclidean algorithm: the GCD of `a` and `b` is the same as the GCD of `b` and the remainder of `a % b`. We repeat this step in a `while` loop until the remainder is 0, and then the other number is the GCD. For instance, for 6 and 15: (6, 15) → (15, 6) → (6, 3) → (3, 0), so the GCD is 3. The function receives two integers and returns an integer. *(+12 points)*

For negative numbers, we first replace both inputs by their absolute values, `a = abs(a)` and `b = abs(b)`, before the loop. The divisors of a number and of its negative are the same, so this always returns a positive GCD. *(+3 points)*

If one input is zero, for example (0, 5), the first step of the loop gives (5, 0) and the loop stops, returning the absolute value of the non-zero number. It also works if the zero is the second input, as the loop does not even start and we return `a`. *(+3 points)*

If both inputs are zero, the loop never starts (`b` is 0) and the function returns `a`, which is 0, as required. *(+2 points)*

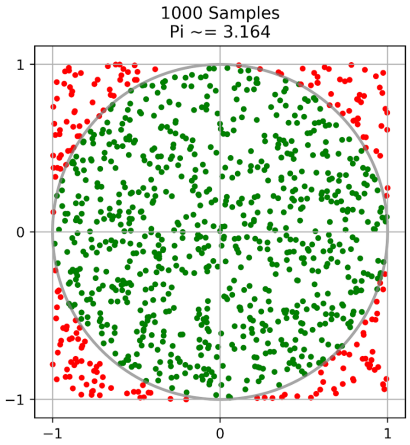
Comment: A valid alternative is a loop that tries every number from `min(a, b)` down to 1 and returns the first one that divides both. It is accepted, but then the zero cases need explicit `if` clauses, and it is much slower.

(Optional, code)

```python
def find_gcd(a: int, b: int) -> int:
    # Work with positive numbers
    a = abs(a)
    b = abs(b)
    # Euclidean algorithm
    while b != 0:
        a, b = b, a % b
    return a
```

6. Approximating Pi with Monte Carlo (20 points)

Consider a square with a circle inside it. We randomly generate points within this square. The image below illustrates this with green points inside the circle and red points outside.



To estimate the value of π, we use the Monte Carlo method. This involves comparing the number of points inside the circle (green) to the total number of points (green and red). The formula for approximating π is:

$$\pi \approx 4 \times \frac{\text{number of blue points}}{\text{total number of points}}$$

Write the `monte_carlo_pi()` function to approximate π. It should:

- Accept `num_points`, the number of random points to create.
- Generate random points in a square with corners (-1, -1) and (1, 1).
- Count how many points fall inside the unit circle $x^2 + y^2 \leq 1$.
- Calculate π following the equation above.
- Return this estimated value of π.

Answer:

The function receives `num_points`, an integer, and returns a float, the estimation of π. *(+2 points)*

First, we generate the random points. Each point has a coordinate x and a coordinate y, both uniformly distributed between -1 and 1. With NumPy, we can create all of them at once with `x = np.random.uniform(-1, 1, num_points)`, and the same for `y`. A loop that creates one point at a time would also be valid. *(+6 points)*

Next, we count how many points are inside the unit circle. The condition `x**2 + y**2 <= 1` applied to the arrays returns a boolean array, with one element per point. Because `True` counts as 1 and `False` as 0, `np.sum()` of that array is the number of points inside the circle. *(+6 points)*

Then we apply the formula, `4 * inside / num_points`. *(+4 points)*

Finally, we return that value (not print it). *(+2 points)*

(Optional, code)

```python
import numpy as np

def monte_carlo_pi(num_points: int) -> float:
    # Random points inside the square with corners (-1, -1) and (1, 1)
    x = np.random.uniform(-1, 1, num_points)
    y = np.random.uniform(-1, 1, num_points)
    # Count the points inside the unit circle
    inside = np.sum(x**2 + y**2 <= 1)
    # Ratio of areas, times 4
    return 4 * inside / num_points
```

Cheatsheet

Given a number named `x`:

- `x % y` returns the remainder of the division $\frac{x}{y}$.
- `x // y` returns the quotient of the division $\frac{x}{y}$.
- `abs(x)` returns the absolute value of `x`.

Given a string named `s`:

- `int(s)` returns an integer. Fails if `s` has invalid characters.
- `float(s)` returns a floating number. Fails if `s` has invalid characters.
- `s.lower()` returns a copy of the string, all in lowercase.
- `s.split(" ")` split a string into a list where each word is a list item.
- `s.count(<pattern>)` returns the number of non-overlapping occurrences of `<pattern>`.
- `s.find(<pattern>, <start>)` finds the first occurrence of the specified `<pattern>`, starting from index `<start>`. Returns `-1` if the pattern is not found.
- `s.replace(<pattern>, <new>)` returns a new string where all instances of `<pattern>` have been replaced with `<new>`.

Given a list named `ls`:

- `ls.append(element)` add element to that list.

Given a list or tuple named `ls`:

- `ls.index(element)` returns the index of element in that list.
- `len(ls)` returns the number of elements in that list.
- `sum(ls)` returns the sum of the elements of that list.
- `sorted(ls)` returns a sorted version of that list.
- `tuple(ls)` / `set(ls)` returns a tuple / set using the elements of that list.
- `", ".join(ls)` returns a string with the elements of the list separated by commas.

Given a set named `a`:

- `a.add(x)` adds the element `x` to the set `a`.
- `a.union(b)` or (`a | b`) returns the union of set `a` with another set `b`.
- `a.intersection(b)` or (`a & b`) returns the intersection of set `a` with another set `b`.
- `a.difference(b)` or (`a - b`) returns the difference of set `a` with another set `b`.

Given a dictionary named `d`:

- `list(d.keys())` returns a list with the keys of `d`.
- `list(d.values())` returns a list with the values of `d`.
- `list(d.items())` returns a list with the tuples (key, value) of `d`.

Some useful build-in functions:

- `type(x)` returns the type of `x` (int, float, list...).
- `range(start, stop)` generates numbers from `start` until `stop`, not including `stop`.
- `input(msg)` is used to take user input, in form of a string.

NumPy Cheatsheet

Importing NumPy:

- `import numpy as np`
- `rng = np.random.default_rng()`

Creating arrays:

- `np.zeros((m, n))` creates an $m \times n$ array filled with zeros.
- `np.ones((m, n))` creates an $m \times n$ array filled with ones.
- `np.arange(start, stop, step)` creates values from `start` to `stop` (exclusive) with a given `step`.
- `np.linspace(start, stop, num)` creates `num` evenly spaced values between `start` and `stop`.

Given a NumPy array `arr`:

- `arr.ndim` returns the number of array dimensions, as a number.
- `arr.shape` returns the shape of an array, as a tuple.

Array operations:

- `np.dot(a, b)` computes the dot product.
- `np.matmul(a, b)` or `a @ b` performs matrix multiplication.
- `np.transpose(a)` or `a.T` transposes the array.

Array manipulation:

- `a.reshape((m, n))` changes the shape of an array.
- `a.flatten()` flattens a multidimensional array into 1D.
- `np.concatenate((a, b), axis=0)` joins arrays along an axis.

- `np.stack(<sequence>)` joins a sequence of arrays along a new axis.
- `np.vstack((a, b))`, `np.hstack((a, b))` vertical/horizontal stacking.
- `np.split(a, n)` splits an array into `n` parts.

Statistics and descriptive analysis:

- `np.mean(a)` mean of all elements.
- `np.median(a)` median of all elements.
- `np.std(a)` standard deviation.
- `np.var(a)` variance.
- `np.min(a)`, `np.max(a)` minimum and maximum.
- `np.argmin(a)`, `np.argmax(a)` indices of minimum and maximum.
- `np.corrcoef(a, b)` correlation coefficient matrix.
- `np.cov(a, b)` covariance matrix.

Linear algebra:

- `np.linalg.inv(a)` inverse of matrix `a`.
- `np.linalg.det(a)` determinant of `a`.
- `np.linalg.eig(a)` eigenvalues and eigenvectors.
- `np.linalg.norm(a)` vector or matrix norm.
- `np.linalg.solve(a, b)` solves $Ax = b$.

Logical operations and conditions:

- `np.where(cond, x, y)` selects elements from `x` or `y` based on condition.
- `np.all(a)`, `np.any(a)` check if all/any elements are True.
- `np.unique(a)` returns sorted unique elements.

Random number generation (using `rng`):

- `rng.random(size)` uniform distribution in $[0, 1)$.
- `rng.integers(low, high, size)` random integers in $[low, high)$.
- `rng.normal(mean, std, size)` normal (Gaussian) distribution.
- `rng.uniform(low, high, size)` uniform distribution in $[low, high)$.
- `rng.choice(a, size)` random sample from array `a`.

Random number generation (legacy API):

- `np.random.randint(low, high, size)` returns random integers from low (inclusive) to high (exclusive).
- `np.random.uniform(low, high, size)` samples from a uniform distribution.
- `np.random.seed(seed)` fixes the seed. After a seed reset, the same set of numbers will appear every time.

Math and trigonometry:

- `np.power(arr1, arr2)` returns the first array elements raised to powers from second array, element-wise.
- `np.sin(arr)` returns the trigonometric sine, assuming angle in radians.
- `np.cos(arr)` returns the trigonometric cosine, assuming angle in radians.
- `np.radians(arr)` converts angles from degrees to radians.
- `np.degrees(arr)` converts angles from radians to degrees.